

CoverageDesk — Course Project Proposal

Eungchan Kang · PSU Access ID: ekk5635

1. Project Choice

Project V — Bring Your Own Domain: CoverageDesk.

A web application that tells an ordinary person what shape a public company is in — how its finances have moved, and how openly it reports — in terms that are easy to understand.

2. Problem Statement

The information is public, but it is not written for the public. Everything a public company must disclose is already free through the SEC's EDGAR system. It arrives as hundreds of pages of prose and raw XBRL facts — a format built for regulators and analysts, not for the person who simply wants to know whether the company they invested in, or work at, is holding up. So an ordinary person asking “*how is this company doing?*” ends up with whatever unverified article happens to be circulating online.

AI is no different. Ask it about a company and it produces a fluent paragraph, but what it is summarizing is that same unsourced material. Where being confidently wrong has a real cost, an answer with no source is not an answer.

Who the users are. The product exists for the **User**: an ordinary individual with no finance training who has invested in public companies, or is simply interested in them. They register the companies they care about and read the results. Two further roles hold different privileges. A **Viewer** reads only the coverages a User has explicitly shared. An **Admin** operates the system — accounts, and the health of the analysis pipeline — with no access to anyone's coverages. Signing in creates a User; the Viewer privilege is granted by a coverage's owner at the moment they share it; Admin is seeded and never self-assigned.

3. Core Features

MUST-HAVE

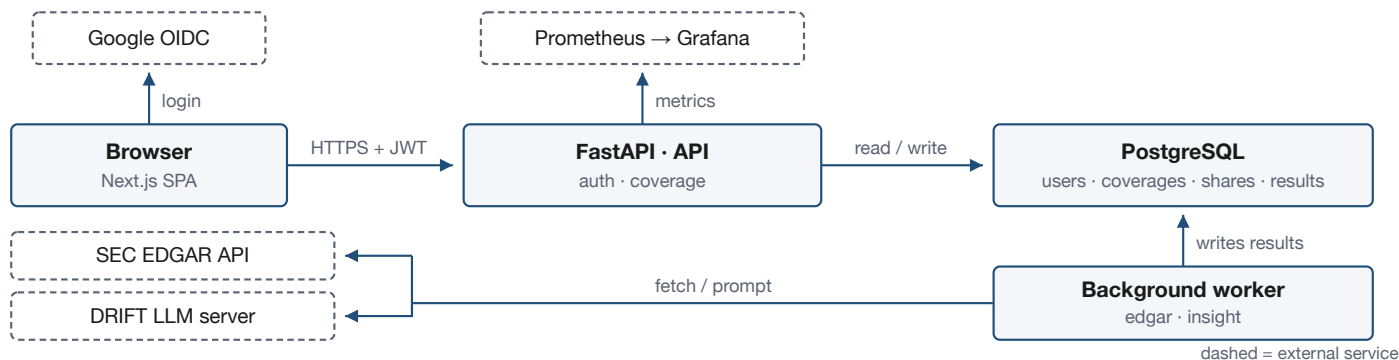
- Coverage CRUD.** A User creates, edits, and deletes a *coverage*: a company plus the reason and the conditions they are watching it for, stored as `id`, `title`, `description`, `status`, and `owner_id`. A coverage moves through `WATCHING` → `ANALYZING` → `READY` → `ARCHIVED`, and every query is scoped by the requester's identity and role.
- Authentication and role-based access control.** Google OAuth 2.0 / OIDC login, JWT-backed sessions, and the User / Viewer / Admin roles enforced in middleware — including the per-coverage share that grants the Viewer privilege.
- Financial and disclosure metrics.** For a coverage's company, retrieve its XBRL facts and filing history from SEC EDGAR and compute two families of signals: financial movement (year-over-year revenue growth, operating margin, debt ratio) and disclosure transparency (filing timeliness, amendment frequency). A company that reports late or restates often is telling you something before its numbers do.
- Plain-language summary with sources.** The summary path sends the computed metrics — and nothing else — to the course DRIFT inference server, and stores the resulting short summary on the coverage together with the filings it was derived from.

NICE-TO-HAVE

- Retrieval over filing text** — chunk and embed disclosure text with `qwen3-embedding` to surface comparable passages across filings.
- New-filing alerts** — detect that a watched company has filed something new, adding a `READY` → `WATCHING` transition so the coverage is analysed again.

4. Architecture Sketch

Modular monolith: one codebase and one image, deployed as two endpoints — the API and a scheduled worker. The worker exists because the inference server allows only a limited number of calls per window, so a summary cannot be generated inside a request. Microservices were considered and rejected: in a seven-week, single-developer project, the cost of running and observing several independently deployed services would consume the time the course spends on testing, containerization, and observability.



Four modules carry the work: **auth** (Google OIDC login, JWT issue and verification, role checks), **coverage** (CRUD, ownership and share scoping, status transitions), **edgar** (EDGAR client and the metric computation, pure and unit-testable), and **insight** (DRIFT client, prompt assembly, and caching of summaries with their sources).

TWO DECISIONS THAT SHAPE THIS DESIGN

- **The model never computes a number.** Ratios and disclosure signals are calculated in **edgar**, which has no model dependency; **insight** receives finished values and may only phrase them. A wrong summary traces to either the arithmetic or the wording — never both.
- **Access is decided at the query layer, not the view layer.** Every read and write of a coverage is scoped by the requester's identity and role before it reaches the database: a User matches on `owner_id`, a Viewer matches on an explicit share, and an Admin reaches no coverage at all. The worker holds a service identity and writes only to the coverage it was given. No caller reaches another user's record by guessing an identifier — the broken-object-level-authorization risk the course asks us to design against.

5. Tech Stack

Layer	Choice
Client	Next.js (React, TypeScript)
Server	FastAPI (Python); the same image also runs the scheduled worker
Database	PostgreSQL
Authentication	Google OAuth 2.0 / OIDC with JWT sessions
External services	SEC EDGAR <code>companyfacts</code> and <code>submissions</code> ; course DRIFT inference server
Testing	pytest (server), Vitest with React Testing Library (client), Playwright (end-to-end)
Delivery & operations	Multi-stage Docker build, Docker Compose, GitHub Actions; structured logging with Prometheus metrics on a Grafana dashboard

6. GitHub Link

<https://github.com/psu-edu/sweng861-capstone-ekk5635>

Public, cloned from the course starter, and initialized with a `README.md` describing the project and the repository structure required by the starter. The weekly assignment repository, `sweng861-crud-ekk5635`, develops the same domain, so the authentication middleware, CI workflow, and test setup built there carry directly into this project.

AI USAGE

An assistant was used to pressure-test the scope of this proposal and to draft wording. The domain, the module boundaries, the separation between computed metrics and generated text, and the decision to keep retrieval out of the Must-Have set are mine, and I revised the draft accordingly.